

The bCCNN as a whole: what it estimates, why early stopping regularises it, how its reserve is judged, and how its uncertainty can be measured

Explainer that joins the companion documents on the model, the fitting procedure and the time-aware split; for `bccnn_calibrate()`, `bccnn_tables()`, `R/metrics.R`, `R/scoring` and the classical wrappers in `R/classical_models.R`, read against Gabrielli, Richman and Wüthrich (2020), Härkönen (2021), Wüthrich and Merz (2019) and England and Verrall (2002)

Prepared for Francis

6 October 2026

Contents

Notation used in this document

1 What this document covers

2 The bCCNN in one formula

3 What the bCCNN estimates

3.1 The target is the mean of each cell

3.2 Balance: a property the chain ladder has and the bCCNN loses

3.3 What the correction can and cannot represent

4 Why 547 parameters on 210 cells do not over-fit: regularisation by early stopping

4.1 The budget on the parameters

4.2 Early stopping as shrinkage towards the chain ladder

4.3 Dropout

5 How the reserve is judged

5.1 The back-test against the simulated truth

5.2 The out-of-time test error

5.3 The classical fits and the scores in `R/scoring`

5.4 The three diagnostic plots

6 Uncertainty of the bCCNN reserve

6.1 The decomposition

6.2 Process variance

6.3 Estimation variance: the parametric bootstrap

6.4 Seed variance and the nagging predictor

6.5 Putting the numbers together

7 The bCCNN among the models of the thesis

8 Where the pieces are in the code

9 Numerical illustration

References

Notation used in this document

The symbols are those of the companions *The mathematics behind the bCCNN* (“companion I”), *The fitting procedure and training of the bCCNN* (“companion II”) and *The time-aware split for the bCCNN* (“companion III”); the lower block is new.

Symbol	Meaning	Section
n, i, j, k	triangle size 20; accident, development and calendar period	2
$\mathcal{O}, \mathcal{F}$	observed cells (210) and future cells (190)	2
$Y_{i,j}, y_{i,j}; R$	payments of a cell; the true outstanding payments $\sum_{\mathcal{F}} Y_{i,j}$	2
$\hat{\mu}_{i,j}^{\text{cc}}, \hat{\phi}_D$	fitted ccODP means and dispersion (companion I, eqs. (4), (18))	2
$\mu_{i,j}(\theta), \theta_0, t^*, \hat{\theta}$	the network’s means, its start, the number of steps, the fitted parameters	2
$\hat{R}^{\text{cc}}, \hat{R}^{\text{bCCNN}}$	the two reserves, sums of the means over $\mathcal{F}$	2
$D(\mathbf{y}, \boldsymbol{\mu}; \mathcal{S})$	Poisson deviance on the cells $\mathcal{S}$	3
η, ρ	RMSprop learning rate and decay, 0.001 and 0.9	4
	New in this document	
$\Gamma_{\theta}(i, j)$	the network’s log-scale correction of the chain ladder, $\log \mu_{i,j}(\theta) - \log \hat{\mu}_{i,j}^{\text{cc}}$	2
$d(y, \mu)$	one summand of the deviance, $\mu - y + y \log(y/\mu)$	3.1
H, λ_r	Hessian of the loss at its minimiser and its eigenvalues (quadratic picture of early stopping)	4.2
MSEP	conditional mean squared error of prediction of a reserve	6
PV, EV	process variance and estimation variance	6
$Y^{*b}, \hat{R}^{*b}$	a bootstrap pseudo-triangle and the reserve refitted on it, $b = 1, \dots, B$	6.3
$\hat{R}^{(s)}, \bar{R}_K$	the reserve for seed s ; the average over K seeds (nagging predictor)	6.4
z	standardised reserve error $(R - \hat{R})/\widehat{\text{se}}$	6.5

“Paper C” is Gabrielli, Richman and Wüthrich (2020); “Härkönen” is Härkönen (2021), the thesis you supplied, cited by her section and equation numbers.

1 What this document covers

Companions I to III each take one piece of the bCCNN apart. This document puts the pieces back together and answers the questions that sit above them: what quantity the bCCNN estimates and why the Poisson deviance is the right criterion for it (Section 3); why a network with 547 parameters can be fitted to 210 cells without reproducing them, that is, what early stopping from the chain-ladder start does mathematically (Section 4); how the resulting reserve is judged in the code and in the papers (Section 5); and how its uncertainty can be measured, which the code does not yet do for the bCCNN but does for the classical models (Section 6). Table 1 is the map.

question	where it is answered
What is the ccODP, why is it the chain ladder, what is the network, what happens at the start	companion I
Which data train, which data stop the training, what the optimiser does, what is random, what the outputs are	companion II
How the time-aware partitions are built, what they guarantee, how they score the procedure out of time	companion III
What the whole thing estimates, why it does not over-fit, how it is judged, how uncertain it is	this document

Table 1: The four explainers.

2 The bCCNN in one formula

Putting companions I and II together, the reported model is the chain ladder with a multiplicative correction learned in t^* gradient steps:

$$\log \mu_{i,j}(\hat{\theta}) = \underbrace{\hat{c} + \hat{\alpha}_i + \hat{\beta}_j}_{\log \hat{\mu}_{i,j}^{\text{cc}}, \text{ the chain ladder}} + \underbrace{\Gamma_{\hat{\theta}}(i, j)}_{\text{learned in } t^* \text{ steps}}, \quad (1)$$

$$\Gamma_{\theta}(i, j) = (w - 1)(\hat{\alpha}_i + \hat{\beta}_j) + (c - \hat{c}) + \langle B, z^{(3)}(i, j) \rangle, \quad \Gamma_{\theta_0} \equiv 0,$$

and the reserve is

$$\hat{R}^{\text{bCCNN}} = \sum_{(i,j) \in \mathcal{F}} \hat{\mu}_{i,j}^{\text{cc}} e^{\Gamma_{\hat{\theta}}(i,j)}, \quad \hat{R}^{\text{cc}} = \sum_{(i,j) \in \mathcal{F}} \hat{\mu}_{i,j}^{\text{cc}}. \quad (2)$$

Everything about the bCCNN is a statement about Γ : it is zero at the start (companion I, Property 1); its first step is a linear combination of ten random features of $(\hat{\alpha}_i, \hat{\beta}_j)$ (Property 2); it is a smooth function of the two scalars $\hat{\alpha}_i$ and $\hat{\beta}_j$ only (companion I, Section 4.4); its size is bounded by the number of steps (Section 4 below); and it is evaluated, for the reserve, at pairs $(\hat{\alpha}_i, \hat{\beta}_j)$ with $i + j > n + 1$ that never occur in the training cells. The relative-difference heat map of the script is $e^{\Gamma} - 1$.

3 What the bCCNN estimates

3.1 The target is the mean of each cell

The ccODP assumption (companion I, eq. (4)) and the network both model the *expected* payment of a cell, $\mathbb{E}[Y_{i,j}]$, with the cells independent. The reserve (2) is therefore an estimate of the expected outstanding payments, $\mathbb{E}[R] = \sum_{\mathcal{F}} \mathbb{E}[Y_{i,j}]$, the actuarial best estimate. Nothing in the bCCNN estimates a distribution, a quantile or a variance; that is why its dispersion needs the separate rule of companion I, Section 8, and its uncertainty the methods of Section 6.

The Poisson deviance is the right training, validation and test criterion for this target, because it is strictly consistent for the mean. For one cell, with $d(y, \mu) = \mu - y + y \log(y/\mu)$,

$$\frac{\partial}{\partial \mu} \mathbb{E}[d(Y, \mu)] = \frac{\partial}{\partial \mu} \left(\mu - \mathbb{E}[Y] \log \mu + \mathbb{E}[Y \log Y] - \mathbb{E}[Y] \right) = 1 - \frac{\mathbb{E}[Y]}{\mu}, \quad (3)$$

which vanishes only at $\mu = \mathbb{E}[Y]$, where the expected loss is minimal (the second derivative $\mathbb{E}[Y]/\mu^2$ is positive). This holds for *any* distribution of Y with that mean, not only an over-dispersed Poisson (Gneiting 2011; Wüthrich and Merz 2023, Chapter 4, where the Poisson deviance is the Bregman divergence of $y \log y$). So: minimising the in-sample deviance aims the network at the cell means; choosing t^* by the validation deviance selects the step whose means are closest, in expectation, to the validation cells' means; and the out-of-sample deviance on the true lower triangle judges the means where they matter. The reserve bias, $\hat{R} - R$, is the same comparison summed over the future cells, and a mean-consistent fit of every cell gives an unbiased reserve, while the converse is not true (cell errors can cancel in the sum). This is why the tables report both.

3.2 Balance: a property the chain ladder has and the bCCNN loses

The ccODP MLE reproduces the observed row and column totals (companion I, eq. (12)); in particular the fitted total over the observed triangle equals the observed total, $\sum_{\mathcal{O}} \hat{\mu}_{i,j}^{\text{cc}} = \sum_{\mathcal{O}} y_{i,j}$. This “balance property” (Wüthrich and Merz 2023, Section 5.1.5; Wüthrich 2020 for networks) is lost after the first gradient step, because $\Gamma \neq 0$ breaks the marginal-total equations: the bCCNN's fitted total over $\mathcal{O}$ is close to, but not equal to, the observed total, and the gap is a bias of the fitted level that propagates into the reserve. The code does not correct it. A one-line correction exists, rescaling the means so that the in-sample total is matched,

$$\tilde{\mu}_{i,j} = \mu_{i,j}(\hat{\theta}) \cdot \frac{\sum_{\mathcal{O}} y_{i,j}}{\sum_{\mathcal{O}} \mu_{i,j}(\hat{\theta})}, \quad (4)$$

which is the same as refitting the output intercept c by its own maximum likelihood equation with the rest of $\hat{\theta}$ held fixed; your neural-network chain-ladder strand has an analogous balance correction (its sensitivity run S4). Whether to apply (4) to the bCCNN is a modelling decision; it is mentioned here because the in-sample total is the first thing to read off `mu_bccnn` when judging a fit.

3.3 What the correction can and cannot represent

Because $z^{(0)}(i, j) = (\hat{\alpha}_i, \hat{\beta}_j)$, the correction is $\Gamma(i, j) = g(\hat{\alpha}_i, \hat{\beta}_j)$ for a smooth surface g on the plane. Three structural consequences follow.

- **Interactions, yes.** The ccODP assumes one development pattern for all accident periods. A surface over (α, β) can bend that pattern differently for different accident periods, which is exactly what the cumulative-development-factor plot of companion I, Section 9, displays: parallel rows for the chain ladder, diverging rows for the bCCNN. This is the effect Paper C and Härkönen report as the gain of the “boosting”.
- **Calendar effects, only indirectly.** A payment inflation acting along the diagonals $k = i + j - 1$ is a function of $i + j$, which g can imitate only to the extent that $\hat{\alpha}$ is monotone in i and $\hat{\beta}$ in j ; there is no calendar input. The chain ladder cannot represent it either, so both models share this blind spot, and a diagonal pattern in the Pearson residuals of *both* models is the signature to look for.
- **Extrapolation into the lower triangle.** The training cells have $i + j \leq n + 1$: large $\hat{\alpha}_i$ (late accident periods) occur in training only with early development periods, and very negative $\hat{\beta}_j$ (late development periods) only with early accident periods. The future cells pair late accident periods with late development periods, a corner of the (α, β) plane with no training support. There g is an extrapolation of a composition of $\tanh$ functions, which saturate, so the extrapolation is bounded but not controlled by data. This is the structural reason for keeping t^* small, for Paper C’s finding that the in-sample validation did not predict the out-of-sample gain for every LoB, and for the out-of-time test partitions of companion III. The short tail of your portfolio helps: most of the reserve sits in the first development periods of the latest accident periods, where the extrapolation is short.

4 Why 547 parameters on 210 cells do not over-fit: regularisation by early stopping

4.1 The budget on the parameters

Härkönen’s count (her eq. (76)) and companion I’s Table 1 agree: 547 trainable numbers, 210 cells. A network of that size can interpolate the training triangle, and a run long enough would do so (Paper C: “after that the algorithm starts to over-fit to the observations”). Three things prevent it in the procedure: the start at the chain ladder, the number of steps, and dropout.

The first two combine into a hard bound. RMSprop with Keras’s rule satisfies $|g_t|/\sqrt{v_t} \leq 1/\sqrt{1-\rho}$ at every step (companion II, eq. (6)), so after t steps every coordinate of θ is within

$$|\vartheta_t - \vartheta_0| \leq \frac{\eta}{\sqrt{1-\rho}} t = 0.00316 t \quad (5)$$

of its start. Through (1), with $|z^{(3)}| \leq 1$ componentwise, the log-scale correction is then bounded by

$$|\Gamma_{\theta}(i, j)| \leq |w_t - 1| |\hat{\alpha}_i + \hat{\beta}_j| + |c_t - \hat{c}| + \sum_{r=1}^{10} |B_{t,r}| \leq 0.00316 t (|\hat{\alpha}_i + \hat{\beta}_j| + 11). \quad (6)$$

For $t^* = 300$ and a cell with $|\hat{\alpha}_i + \hat{\beta}_j| \leq 5$, (6) gives $|\Gamma| \leq 15$, which is not a useful number: the bound is a worst case in which every coordinate moves in the same direction at the maximal rate at every step. In practice the gradients change sign and the realised distance is smaller (Section 9: in the toy run the largest move of any of the 547 coordinates after 407 steps was **0.31**, a quarter of the bound **1.29**, and the ten output weights B that carry the correction moved by at most **0.03**). What (5)-(6) do establish is that the

number of steps is the control on the model’s distance from the chain ladder, and that no run of t steps can produce a correction larger than linear in t .

4.2 Early stopping as shrinkage towards the chain ladder

The textbook argument (Goodfellow et al. 2016, Section 7.8; Bishop 1995) makes the budget precise in a quadratic picture. Replace the loss near its minimiser θ^* (the interpolating network) by its second-order expansion $\ell(\theta) \approx \ell(\theta^*) + \frac{1}{2}(\theta - \theta^*)^\top H(\theta - \theta^*)$ with H positive semi-definite, eigenvalues $\lambda_r \geq 0$ and eigenvectors u_r . Plain gradient descent with learning rate η from θ_0 gives, in the eigen-coordinates $\xi_r = u_r^\top(\theta - \theta_0)$ and $\xi_r^* = u_r^\top(\theta^* - \theta_0)$,

$$\xi_{t,r} = (1 - (1 - \eta\lambda_r)^t) \xi_r^*. \quad (7)$$

Ridge regression from the same start, $\min_\theta \ell(\theta) + \frac{\kappa}{2}\|\theta - \theta_0\|^2$, gives $\xi_r = \frac{\lambda_r}{\lambda_r + \kappa} \xi_r^*$. The two agree when $(1 - \eta\lambda_r)^t = \kappa/(\lambda_r + \kappa)$, and for $\eta\lambda_r \ll 1$ and large t both sides are approximately $e^{-\eta\lambda_r t}$ and $1/(1 + \lambda_r/\kappa)$, so

$$\kappa \approx \frac{1}{\eta t}. \quad (8)$$

In words: t gradient steps from the chain-ladder start are approximately a ridge fit penalised towards the chain ladder with penalty $1/(\eta t)$. Directions of the parameter space in which the data pull strongly (large λ_r : the correction is well supported by many cells) are learned almost completely within a few hundred steps, $1 - e^{-\eta\lambda_r t} \approx 1$; directions in which the data pull weakly (small λ_r : idiosyncratic cells, the extrapolation corner of Section 3.3) are barely moved, $\eta\lambda_r t \ll 1$. The 547 parameters are therefore not 547 free degrees of freedom; the effective number is the number of eigen-directions with $\eta\lambda_r t^* \gtrsim 1$, and it grows with t^* . RMSprop rescales each coordinate’s learning rate by its running gradient size, so (8) holds per coordinate with an effective η of order $\eta/\sqrt{v}$, but the picture is the same. This is the precise sense in which “the number of gradient descent steps” is the single hyperparameter Paper C and Härkönen tune, and why $t^* = 0$ is the chain ladder and $t^* \rightarrow \infty$ the interpolating network.

4.3 Dropout

Dropout (companion I, Section 4.5) is the third regulariser: with 10% of the hidden units removed at random at each step, the gradient the network follows is that of a random sub-network, and features that depend on particular co-adapted units are penalised. Härkönen’s phrasing is the standard one: units “have to learn to co-operate with a random sample of other neurons”. Its cost is visible in the curves: the recorded validation deviance is noisy even though it is computed without dropout, because the parameters themselves jitter, and the exact minimum of companion II, eq. (8), can land on a lucky step. Paper C fixed the rate at 10% after finding that it “provided stable predictive models over several runs” (Gabrielli’s thesis version of Paper C, §3.3.2); Härkönen used 20% in the double network.

5 How the reserve is judged

5.1 The back-test against the simulated truth

`bccnn_tables()` reports, for the chain ladder and the bCCNN, the reserve, the bias $\hat{R} - R$ and the percentage bias, the in-sample deviance on $\mathcal{O}$, the out-of-sample deviance on $\mathcal{F}$, and the dispersion

(companion II, Section 9). The by-origin table splits the bias by accident period, and the script plots it. Reading them together:

- **In-sample deviance** falls with t^* by construction (Paper C’s Table 3, “decrease of in-sample loss”); a large decrease is not a merit, and it is the first symptom of over-fitting when the out-of-sample numbers do not follow.
- **Out-of-sample deviance on $\mathcal{F}$** is the cell-level criterion the model was trained for, evaluated where it was not trained; it penalises a model that gets the total right by cancelling errors.
- **Bias** is what the reserve is for. Paper C’s Table 2 and Härkönen’s Table 3 are tables of percentage biases; the latter shows biases between 0.07% and −9% across six LoBs for the simple network and sign changes when the halves are swapped, so a single bias on a single triangle is a weak basis for a verdict, and the by-origin biases and the out-of-sample deviance are needed to see whether the correction is systematic.
- **The payments after development period n** are reported but modelled by neither model: the comparison is reserve to period n against truth to period n .

5.2 The out-of-time test error

Under the rolling origin, companion III’s test error $\bar{D}$ (eq. (10) there) is the only evaluation available *before* the truth is used: the deviance per test cell of the whole procedure replayed at valuation years 15 and 18, next to the chain ladder at those dates. It is reported on one seed; Al-Mudafer et al. average it over several initialisations before comparing designs.

5.3 The classical fits and the scores in R/scoring

The repository fits the same triangle with `fit_chainladder()` (the chain-ladder factors, no standard error), `fit_mack()` (Mack 1993: the same point estimate with a distribution-free standard error) and `fit_glm_reserve()` (`glmReserve()` with `var.power = 1`, the same ccODP as companion I, with a standard error by the delta method or by England and Verrall’s bootstrap). The script asserts that the ccODP of the bCCNN and the chain ladder agree on the reserve; the Mack and GLM fits therefore also share the bCCNN’s *starting point*, and differ from it only in having a standard error.

`score_total(reserve, se, truth, sims)` in R/scoring scores a total reserve: the error and percentage error; the standardised error $z = (R - \hat{R})/\widehat{se}$ and its normal probability integral transform; whether the truth lies in the 95% interval; the continuous ranked probability score, from a normal predictive distribution with mean $\hat{R}$ and standard deviation $\widehat{se}$,

$$\text{CRPS}(\hat{R}, \widehat{se}; R) = \widehat{se} \left(z(2\Phi(z) - 1) + 2\varphi(z) - \frac{1}{\sqrt{\pi}} \right), \quad (9)$$

or from simulations when they are supplied; and the quantile scores at the 75th and 95th percentiles. As the code stands, the bCCNN has no $\widehat{se}$ and no simulations, so only the first two of these are available for it. Section 6 is about supplying the rest.

5.4 The three diagnostic plots

The relative difference $e^\Gamma - 1$ shows *where* the network moved the chain ladder and by how much, over the whole square. The cumulative development factors show the same thing as development patterns by accident period. The Pearson residuals on the lower triangle, with each model’s own $\hat{\phi}$ and a common

colour scale, show *whether* the move was in the right direction cell by cell; a pattern along the diagonals is the calendar-effect signature of Section 3.3, and a pattern in the latest accident periods is the extrapolation corner.

6 Uncertainty of the bCCNN reserve

6.1 The decomposition

For a reserve estimate $\hat{R}$ computed from the observed triangle, the conditional mean squared error of prediction is (England and Verrall 2002; Härkönen, eq. (78))

$$\text{MSEP}(\hat{R}) = \mathbb{E}[(R - \hat{R})^2 | \mathcal{D}] = \underbrace{\text{Var}(R | \mathcal{D})}_{\text{process variance}} + \underbrace{(\mathbb{E}[R | \mathcal{D}] - \hat{R})^2}_{\text{estimation error}}, \quad (10)$$

the second term being replaced in practice by its expectation over the sampling of the triangle, the estimation variance $\text{EV} = \text{Var}(\hat{R})$. Model error, the possibility that neither the ccODP nor the bCCNN is the true mean structure, is outside (10); Härkönen's Section 5 makes the point that "a model with large relative bias may still achieve low MSEP" for exactly this reason, so (10) is read next to the back-test bias, not instead of it.

6.2 Process variance

Under the ODP assumption on the future cells, independent with $\text{Var}(Y_{i,j}) = \phi \mu_{i,j}$,

$$\text{PV} = \text{Var}(R | \mathcal{D}) = \phi \sum_{(i,j) \in \mathcal{F}} \mu_{i,j}, \quad \widehat{\text{PV}}^M = \hat{\phi}^M \hat{R}^M, \quad (11)$$

for each model M with its own dispersion: $\hat{\phi}_D$ for the chain ladder (companion I, eq. (18)) and Paper C's reduced $\hat{\phi}^{\text{bCCNN}}$ (companion I, eq. (36)). Härkönen's eq. (80) is (11) with claim counts as exposures. The process standard deviation $\sqrt{\phi \hat{R}}$ is the irreducible part: it is there even if the means were known exactly.

6.3 Estimation variance: the parametric bootstrap

For the ccODP, `fit_glm_reserve()` gives EV by the delta method (`mse.method = "formula"`: $\widehat{\text{EV}} = \sum_{\mathcal{F}} \sum_{\mathcal{F}} \hat{\mu}_{i,j} \hat{\mu}_{k,l} \widehat{\text{Cov}}(\hat{\eta}_{i,j}, \hat{\eta}_{k,l})$, England and Verrall 1999) or by the bootstrap ("bootstrap"). The bCCNN has no formula, but it has a procedure that can be rerun on pseudo-data, which is all a bootstrap needs. Härkönen (§2.4, eqs. (82)-(83)) uses the parametric form: simulate pseudo-triangles from the fitted over-dispersed Poisson, using $Y/\phi \sim \text{Poisson}(\mu/\phi)$. Written for the bCCNN:

1. For $b = 1, \dots, B$: draw a pseudo upper triangle $Y_{i,j}^{*b} = \hat{\phi} \cdot \text{Poisson}(\hat{\mu}_{i,j}/\hat{\phi})$ on $\mathcal{O}$, from a generating model $(\hat{\mu}, \hat{\phi})$: the ccODP (the null hypothesis that the chain ladder is right) or the bCCNN itself.
2. Refit the whole procedure on Y^{*b} : the ccODP, the start, and exactly t^* steps with the configured seed (or a fresh seed per replicate, which folds the seed variance of Section 6.4 into the result); read off $\hat{R}^{*b}$.
3. $\widehat{\text{EV}} = \frac{1}{B-1} \sum_b (\hat{R}^{*b} - \bar{R}^*)^2$, and $\widehat{\text{MSEP}} = \widehat{\text{PV}} + \widehat{\text{EV}}$.

Keeping t^* fixed across replicates treats the number of steps as part of the model, as Paper C does when it uses one t^* for all LoBs; re-choosing t^* inside each replicate would also capture the variability of the early-stopping rule, at the cost of a validation run per replicate. The non-parametric alternative of England and Verrall (2002), resampling the scaled Pearson residuals of the ccODP to build pseudo-triangles, is what `glmReserve(mse.method = "bootstrap")` does and would serve the bCCNN equally. `config.yml` already carries `bootstrap: iterations.NN: 10`; each replicate costs one `glm()` and t^* full-batch steps, a few seconds. A full predictive distribution, for the PIT and CRPS of `score_total()`, adds a fourth step: draw the future cells $Y_{i,j}^{*b}, (i, j) \in \mathcal{F}$, from $ODP(\mu_{i,j}^{*b}, \hat{\phi})$ and record R^{*b} as a simulated outcome.

6.4 Seed variance and the nagging predictor

A network has a source of estimation variance a GLM does not have: the seed. Härkönen refitted the double network for 20 seeds at the chosen number of epochs and found biases spread over several percentage points, concluding that “it might be beneficial to fit several models with different seeds and then choose the average prediction” (§3.2 and §5); Al-Mudafer et al. ensemble five fits. Richman and Wüthrich (2020) call the average over seeds the nagging predictor,

$$\bar{R}_K = \frac{1}{K} \sum_{k=1}^K \hat{R}^{(s_k)}, \quad \text{Var}(\bar{R}_K) \approx \frac{\sigma_s^2}{K} + (\text{variance common to all seeds}), \quad (12)$$

where σ_s^2 is the across-seed variance of a single fit. Averaging removes the seed-specific part of the error at rate $1/K$ and leaves the part that comes from the data, which is the estimation variance of Section 6.3 proper. In the bCCNN the seed enters only through the hidden-layer initialisation and the dropout masks (companion II, Section 6); the start and the ccODP are the same for every seed, so σ_s measures how much the *correction* Γ depends on the random features it was built from. A seed study is a loop over `seed` in `bccnn_calibrate()` and a mean of the reserves; your neural-network chain-ladder strand already does this with 20 seeds.

6.5 Putting the numbers together

With $\widehat{\text{se}} = \sqrt{\text{MSEP}}$ from (10)-(11) and the bootstrap, `score_total()` gives the bCCNN the same report as the classical fits: the standardised error z , whose magnitude says whether the back-test bias is within the model’s own uncertainty; the interval coverage; and the CRPS (9), which rewards a reserve that is both close to the truth and honest about its uncertainty. For a reserve that is a sum of means, the right comparison across models is then no longer “whose bias is smaller” but “whose predictive distribution scores better”, which is where the dispersion rule of companion I, Section 8, with its `max(0, .)` cut, becomes consequential: a bCCNN whose validation decrease was large is given a *smaller* process variance than the chain ladder, and that is a claim the back-test should be allowed to contradict.

7 The bCCNN among the models of the thesis

model	mean structure	fitted by	uncertainty	in the repository
chain ladder / Mack	$\hat{C}_{i,n} = C_{i,n+1-i} \prod f_k$	volume-weighted factors	Mack's distribution-free standard error (process + estimation)	<code>fit_chainladder()</code> , <code>fit_mack()</code>
ccODP GLM	$\exp\{c + \alpha_i + \beta_j\}$, same reserve as the chain ladder	quasi-Poisson IWLS	delta method or bootstrap (England and Verrall)	<code>fit_glm_reserve()</code> , <code>fit_odp_glm()</code>
bCCNN (Paper C)	ccODP $\times e^\Gamma$, Γ a network on $(\hat{\alpha}_i, \hat{\beta}_j)$, started at 0	t^* RMSprop steps, early-stopped	none in the code; Section 6	<code>bccnn_calibrate()</code>
double network (Gabrielli 2020; Härkönen §2.3.4)	counts and amounts jointly, with reporting and payment delay, attention over past counts	the same kind of procedure, scaled losses	bootstrap (Härkönen §2.4)	not in this repository
neural network chain ladder (Wüthrich 2018)	factors $f_{j-1}(x)$ depending on claim features	one network per development period, chain-ladder start	nagging over seeds	your <code>R/nn_chain_ladder.R</code> strand

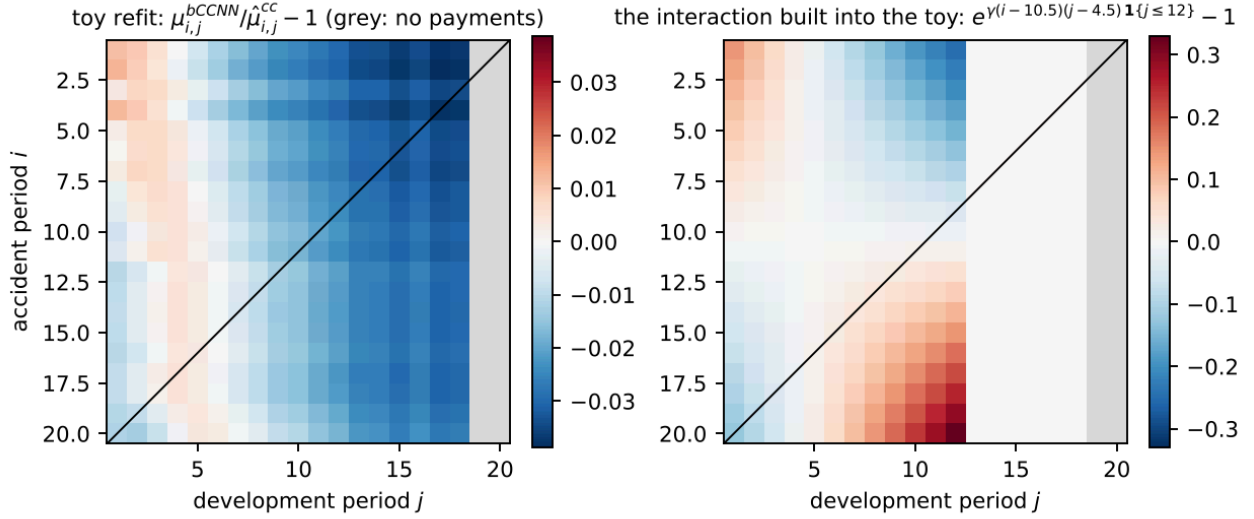
Table 2: Where the bCCNN sits. The first three rows share the same point of departure, the chain-ladder reserve.

8 Where the pieces are in the code

topic	where
the correction Γ and the two reserves, (1)-(2)	<code>bccnn_model()</code> , <code>fit_bccnn()</code> , <code>reserve_by_origin()</code> ; the heat map <code>plot_relative_difference(res\$odp\$mu,</code> <code>res\$nn\$mu)</code>
deviance as the criterion, Section 3.1	<code>compile(loss = "poisson");</code> <code>poisson_deviance()</code> ; the <code>vali</code> , <code>test</code> , <code>truth</code> columns
balance (4)	not in the code; <code>sum(tabs\$mu_bccnn[upper</code> <code>cells])</code> against <code>sum(sets\$upper)</code> shows the gap
the budget (5)-(6)	<code>optimizer_rmsprop(learning_rate = 0.001,</code> <code>rho = 0.9)</code> , <code>max_epochs</code> , <code>epochs</code> in <code>config.yml</code>
dropout, Section 4.3	<code>layer_dropout(rate = 0.1)</code> after each hidden layer
the back-test, Section 5.1	<code>bccnn_tables()</code> : rows “bias”, “in-sample loss”, “out- of-sample loss”, “payments after the last development period”; <code>by_origin</code> ; <code>plot_bias_by_origin()</code>
the out-of-time test error, Section 5.2	<code>bccnn_rolling_origin()</code> \$ <code>test_error</code> ; <code>bccnn_tables()</code> \$ <code>rolling_origin</code>
classical fits and scores, Section 5.3	<code>fit_chainladder()</code> , <code>fit_mack()</code> , <code>fit_glm_reserve()</code> in <code>R/classical_models.R</code> ; <code>score_total()</code> , <code>crps_norm()</code> , <code>qscore()</code> in <code>R/scoring</code>
process variance (11)	$\hat{\phi}$ from <code>res\$phi</code> , reserves from <code>res\$odp\$reserve</code> , <code>res\$nn\$reserve</code> ; not yet multiplied in the code
bootstrap, Section 6.3	not yet in the code for the bccnn; <code>fit_glm_reserve(mse.method = "bootstrap")</code> for the ccODP; <code>config.yml</code> <code>bootstrap:</code> <code>iterations.NN</code>
seed study, Section 6.4	a loop over <code>seed =</code> in <code>bccnn_calibrate()</code> ; <code>cfg\$seed</code>

9 Numerical illustration

The numbers below come from the same synthetic run as companion II, Section 14 (a 20×20 triangle with a mild interaction the chain ladder cannot represent, $t^* = 407$ chosen on the claims split, refit on the full triangle). They illustrate the formulas of this document; they say nothing about your portfolio.



Left: the learned correction $e^\Gamma - 1$ of the synthetic refit over the whole square (the line is the latest diagonal; grey columns are development periods without payments, where the ccODP effect is $-\infty$ and the ratio is meaningless). Right: the interaction the toy was built with, $e^{\gamma(i-10.5)(j-4.5)\mathbf{1}\{j \leq 12\}} - 1$, the multiplicative departure from a cross-classified structure. After 407 steps the correction is within a few per cent, varies mostly with the development period and only mildly with the accident period, and is carried smoothly into the lower triangle: the network has moved a little way from the chain ladder, in the direction the data pull, and has not reproduced the designed interaction, whose strongest cells (late accident periods at development periods 8 to 12) lie in or near the extrapolation corner.

quantity	value
steps t^* ; bound (5) on any coordinate, $0.00316 t^*$	407; 1.29
realised distance from the start, $\max_{\vartheta} \hat{\vartheta} - \vartheta_0 $: over all 547 parameters; over the output weights B ; w ; c	0.313 (a hidden-layer weight); 0.026; 0.008; 0.003
realised correction, $\max_{i,j} \Gamma_{\hat{\theta}}(i, j) $ over the development periods with payments; over their future cells	0.039; 0.038
reserves: chain ladder, bCCNN, truth	1227.3, 1218.9, 1438.8
dispersions: $\hat{\phi}_D, \hat{\phi}^{bCCNN}$	0.314, 0.307
process standard deviation $\sqrt{\hat{\phi} \hat{R}}(11)$: chain ladder, bCCNN	19.6, 19.3
parametric bootstrap, $B = 20$ pseudo-triangles from the ccODP, $t^* = 407$ fixed, same seed: estimation s.d. of the chain-ladder reserve, of the bCCNN reserve; correlation of the two	26.1, 26.1; 1.00
$\sqrt{\text{MSEP}}(10)$: chain ladder, bCCNN	32.6, 32.4
seed study, $K = 10$ seeds, $t^* = 407$: mean, s.d., range of the bCCNN reserve	1231.0, 7.7, 1221.0 to 1242.1
nagging predictor $\bar{R}_{10}(12)$	1231.0

Table 3: The quantities of Sections 4 and 6 on the synthetic run (amounts in millions of the synthetic data).

Four readings. The realised distance from the start is a quarter of the bound (5) for the most-moved coordinate and far less for the output weights that carry the correction: the gradients change sign, and the network settles much closer to the chain ladder than the budget allows, which is the shrinkage picture of

Section 4.2. The bootstrap, which generates pseudo-triangles from the chain ladder, finds the estimation standard deviation of the bCCNN reserve equal to the chain ladder's to three digits and the two reserves correlated at **0.998** across pseudo-triangles: when the chain ladder is the truth, the correction learned in 407 steps is noise that follows the chain ladder of each pseudo-triangle, and it adds no variance of its own at a fixed seed. The variance the network does add is the seed spread, **7.7** on a reserve of about **1230** (0.6%), about a third of the bootstrap standard deviation; the nagging average over ten seeds, **1231.0**, removes most of it. And both models' biases on this toy, **−211** and **−220**, are six to seven times their $\sqrt{\text{MSEP}}$ of about **32**: the interaction grows into the lower triangle, and neither the chain ladder nor a correction that must extrapolate from $(\hat{\alpha}_i, \hat{\beta}_j)$ pairs it has not seen can know that. That is model error, outside (10), and the back-test bias is the only number that sees it.

References

- Al-Mudafer, M. T., Avanzi, B., Taylor, G. and Wong, B. (2021). Stochastic loss reserving with mixture density neural networks. arXiv:2108.07924.
- Bishop, C. M. (1995). Regularization and complexity control in feed-forward networks. Proceedings ICANN'95, 141-148.
- England, P. D. and Verrall, R. J. (1999). Analytic and bootstrap estimates of prediction errors in claims reserving. Insurance: Mathematics and Economics 25(3):281-293.
- England, P. D. and Verrall, R. J. (2002). Stochastic claims reserving in general insurance. British Actuarial Journal 8(3):443-518.
- Gabrielli, A., Richman, R. and Wüthrich, M. V. (2020). Neural network embedding of the over-dispersed Poisson reserving model. Scandinavian Actuarial Journal 2020(1):1-29. ("Paper C".)
- Gabrielli, A. (2020). A neural network boosted double over-dispersed Poisson claims reserving model. ASTIN Bulletin 50(1):25-60.
- Gneiting, T. (2011). Making and evaluating point forecasts. Journal of the American Statistical Association 106(494):746-762.
- Goodfellow, I., Bengio, Y. and Courville, A. (2016). Deep Learning. MIT Press. Section 7.8.
- Härkönen, V. (2021). On claims reserving with machine learning techniques. Master thesis 2021:4, Mathematical Statistics, Stockholm University.
- Mack, T. (1993). Distribution-free calculation of the standard error of chain ladder reserve estimates. ASTIN Bulletin 23(2):213-225.
- Richman, R. and Wüthrich, M. V. (2020). Nagging predictors. Risks 8(3):83.
- Wüthrich, M. V. (2018). Neural networks applied to chain-ladder reserving. European Actuarial Journal 8:407-436.
- Wüthrich, M. V. (2020). Bias regularization in neural network models for general insurance pricing. European Actuarial Journal 10:179-202.
- Wüthrich, M. V. and Merz, M. (2019). Editorial: Yes, we CANN! ASTIN Bulletin 49(1):1-3.
- Wüthrich, M. V. and Merz, M. (2023). Statistical Foundations of Actuarial Learning and its Applications. Springer.